KON429E Autonomous Vehicles
Homework 3 Report

Stanley Path Tracking Algorithm in Simulink

Student Name: *Atakan Kuşçu*
Student ID: *040210520*

June 2, 2026

Abstract

This report presents the implementation and evaluation of the Stanley path tracking algorithm for a kinematic bicycle vehicle model in Simulink. The vehicle is assumed to move with a constant longitudinal velocity while tracking a waypoint-based reference path. The front axle center is used as the reference point for the Stanley controller. The steering command is limited to ± 0.5236 rad, corresponding to $\pm 30^\circ$. The model is simulated using a fixed-step solver with a sampling time of $T_s = 0.05$ s. First, the controller is tested using the nominal Stanley gain $k = 1.0$. Then, the effect of the Stanley gain is investigated by comparing the tracking results for $k = 0.1$, $k = 0.4$, $k = 1.0$, and $k = 10.0$. The results show that low gain values produce smoother but less accurate tracking, whereas very high gain values improve convergence but may introduce aggressive and oscillatory steering behavior.

Contents

1 Introduction **3**

2 Vehicle Model **3**

3 Stanley Path Tracking Controller **4**

4 Waypoint Segment Selection **4**

5 Simulink Implementation **5**

6 Simulation Results **6**
6.1 Part (a): Nominal Stanley Gain 6
6.2 Part (b): Effect of Stanley Gain 6
6.2.1 Trajectory Results for Individual Gain Values 7
6.2.2 Combined Trajectory Comparison 9

7 Error and Control Signal Analysis **9**
7.1 Cross-Track Error 9
7.2 Heading Error 10
7.3 Steering Angle 11

8 Performance Metrics **12**

9 Discussion **13**

10 Conclusion **14**

A MATLAB Function Block Code **15**

B Part (b) MATLAB Analysis Script **17**

1 Introduction

Path tracking is a fundamental problem in autonomous vehicle control. The main objective is to generate a steering command that allows the vehicle to follow a desired reference path while minimizing lateral and heading errors. In this homework, the Stanley path tracking algorithm is implemented in Simulink and tested on a waypoint-based path.

The reference path is defined by consecutive waypoints. The path starts from $(0, 0)$ and ends at $(70, 30)$. The vehicle is modeled using a simple kinematic bicycle model with constant longitudinal velocity. The given vehicle and controller parameters are

$$L = 2.2 \text{ m}, \qquad v = 5.0 \text{ m/s}, \qquad k = 1.0, \tag{1}$$

where L is the wheelbase, v is the vehicle velocity, and k is the Stanley gain.

The main goals of the homework are:

- to implement a kinematic bicycle model in Simulink,
- to implement the Stanley path tracking algorithm,
- to use the front axle center as the vehicle reference point,
- to limit the steering angle to ± 0.5236 rad,
- to simulate the model using a fixed-step solver with $T_s = 0.05$ s,
- to compare the effect of different Stanley gain values.

2 Vehicle Model

The vehicle is represented using a kinematic bicycle model. The rear axle position and vehicle heading are selected as the system states:

$$\mathbf{x} = \begin{bmatrix} x_r & y_r & \psi \end{bmatrix}^T, \tag{2}$$

where x_r and y_r denote the rear axle coordinates, and ψ denotes the vehicle heading angle.

The kinematic bicycle model is given by

$$\dot{x}_r = v \cos(\psi), \tag{3}$$
$$\dot{y}_r = v \sin(\psi), \tag{4}$$
$$\dot{\psi} = \frac{v}{L} \tan(\delta), \tag{5}$$

where δ is the front steering angle.

Since the Stanley controller uses the front axle center as the reference point, the front axle coordinates are calculated as

$$x_f = x_r + L \cos(\psi), \tag{6}$$
$$y_f = y_r + L \sin(\psi). \tag{7}$$

Thus, the Stanley controller computes the lateral error using the point (x_f, y_f) instead of the rear axle position.

3 Stanley Path Tracking Controller

The Stanley controller generates the steering command using two main error components:

- heading error, θ_e,
- front axle cross-track error, e_f.

For the active path segment between two consecutive waypoints P_i and P_{i+1}, the segment direction is

$$\mathbf{t} = \frac{P_{i+1} - P_i}{\|P_{i+1} - P_i\|}. \tag{8}$$

The desired path heading angle is

$$\psi_p = \tan^{-1}\left(\frac{t_y}{t_x}\right). \tag{9}$$

The heading error is then calculated as

$$\theta_e = \text{wrapToPi}(\psi_p - \psi). \tag{10}$$

The cross-track error is calculated by projecting the front axle point onto the active path segment. The signed lateral error is obtained using the normal direction of the segment. With the sign convention used in this implementation, the steering command is computed as

$$\delta = \theta_e - \tan^{-1}\left(\frac{ke_f}{v}\right). \tag{11}$$

Finally, the steering command is saturated as

$$\delta = \text{sat}(\delta, -0.5236, 0.5236). \tag{12}$$

The steering limit corresponds to $\pm 30^\circ$.

4 Waypoint Segment Selection

The reference path is defined as an ordered set of waypoints. At each simulation step, the controller selects the active segment between two consecutive waypoints. The vehicle is not allowed to return to previous segments. Therefore, a persistent active segment index is used inside the MATLAB Function block.

The segment selection algorithm follows these rules:

1. The current active segment index is initialized as $i = 1$.
2. The front axle point is projected onto candidate path segments.
3. The closest segment ahead of the current segment is selected.
4. If the vehicle passes the end of the active segment or becomes sufficiently close to the next waypoint, the segment index is increased.
5. The segment index is never decreased.

This logic ensures that the vehicle follows the waypoint path in the given order.

5 Simulink Implementation

The Simulink model consists of two main subsystems:

- the Stanley controller block,
- the kinematic bicycle model block.

The Stanley controller is implemented using a MATLAB Function block. The controller inputs are

$$x_r,\ y_r,\ \psi,\ k,\ v,\ L,\ \text{waypoints}. \tag{13}$$

The controller outputs are

$$\delta,\ \text{seg_idx},\ e_f,\ \theta_e,\ x_f,\ y_f. \tag{14}$$

The steering command δ is then applied to the kinematic bicycle model. The model states are integrated using a fixed-step simulation with sampling time

$$T_s = 0.05\ \text{s}. \tag{15}$$

The initial vehicle heading is selected according to the first waypoint segment:

$$\psi_0 = \tan^{-1}\left(\frac{y_2 - y_1}{x_2 - x_1}\right). \tag{16}$$

Since the front axle center starts at the first waypoint, the rear axle initial position is selected as

$$x_{r,0} = x_1 - L\cos(\psi_0), \tag{17}$$
$$y_{r,0} = y_1 - L\sin(\psi_0). \tag{18}$$

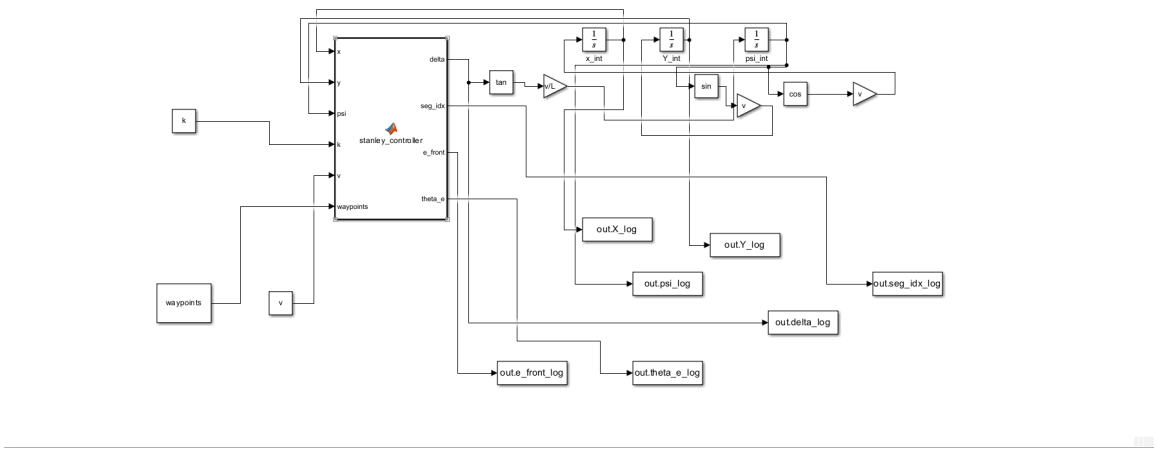


Figure 1: Simulink implementation of the Stanley controller and kinematic bicycle model.

6 Simulation Results

6.1 Part (a): Nominal Stanley Gain

In the first simulation, the Stanley gain is selected as

$$k = 1.0. \tag{19}$$

The resulting vehicle trajectory is shown in Fig. 2. The vehicle successfully follows the waypoint path from the starting point to the final waypoint.

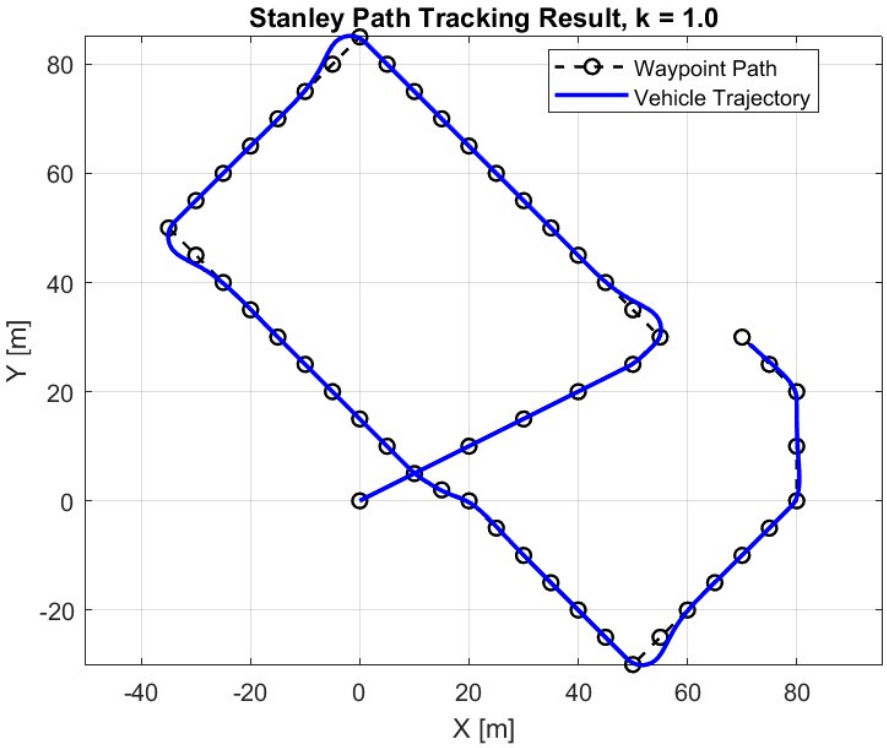


Figure 2: Path tracking result for the nominal Stanley gain $k = 1.0$.

6.2 Part (b): Effect of Stanley Gain

In the second part, the Stanley gain is changed as

$$k = 0.1,\ 0.4,\ 1.0,\ 10.0. \tag{20}$$

The resulting trajectories are compared to evaluate the effect of the gain parameter on path tracking performance.

6.2.1 Trajectory Results for Individual Gain Values

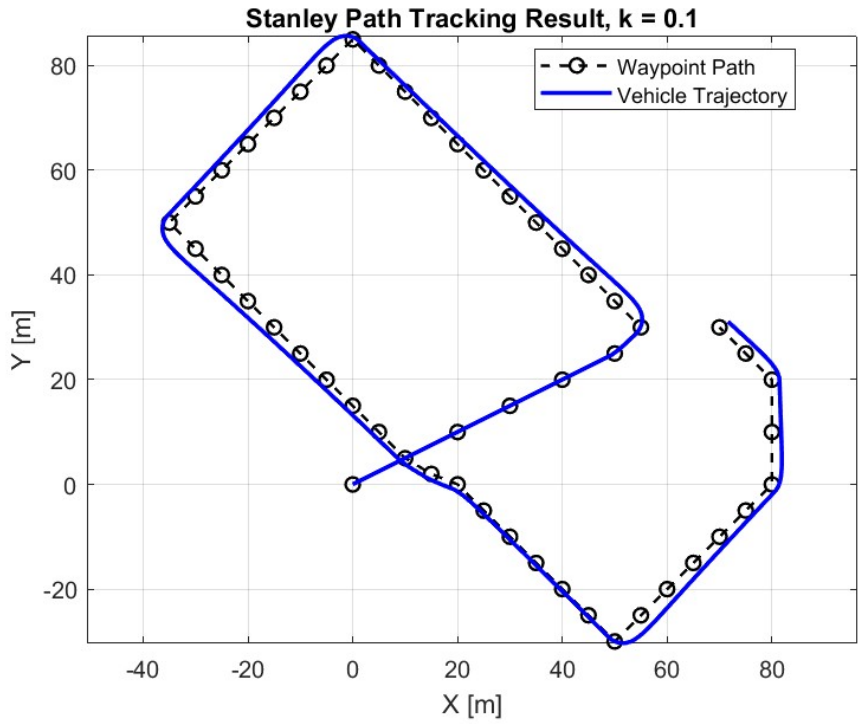


Figure 3: Path tracking result for $k = 0.1$.

For $k = 0.1$, the vehicle follows the path smoothly. However, the cross-track correction term is weak due to the small gain value. As a result, larger lateral deviations are observed, especially around curved sections and waypoint transitions.

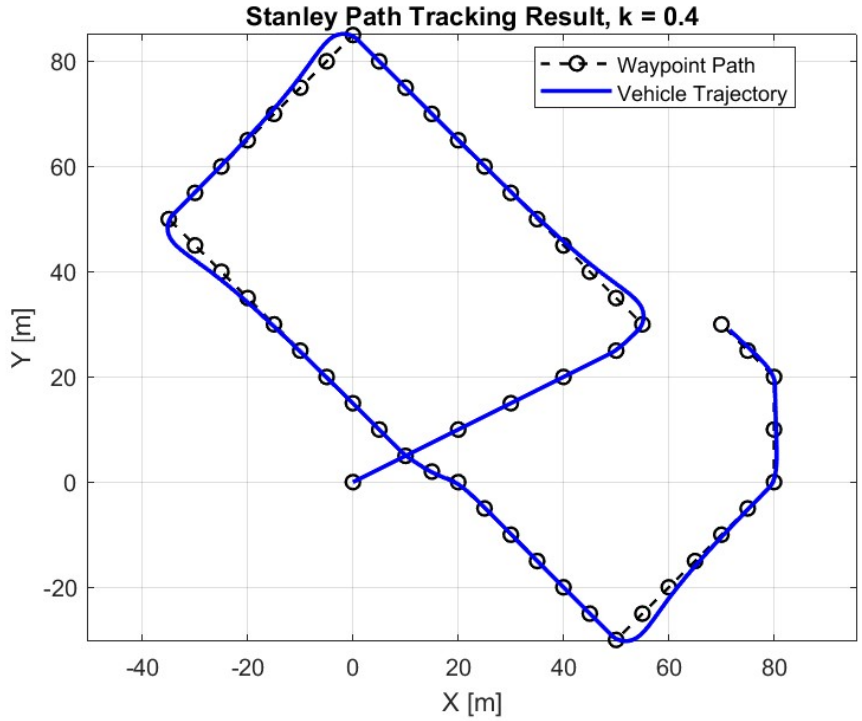


Figure 4: Path tracking result for $k = 0.4$.

For $k = 0.4$, the vehicle tracks the reference path more accurately than in the $k = 0.1$ case. The controller remains smooth while reducing the lateral deviation.

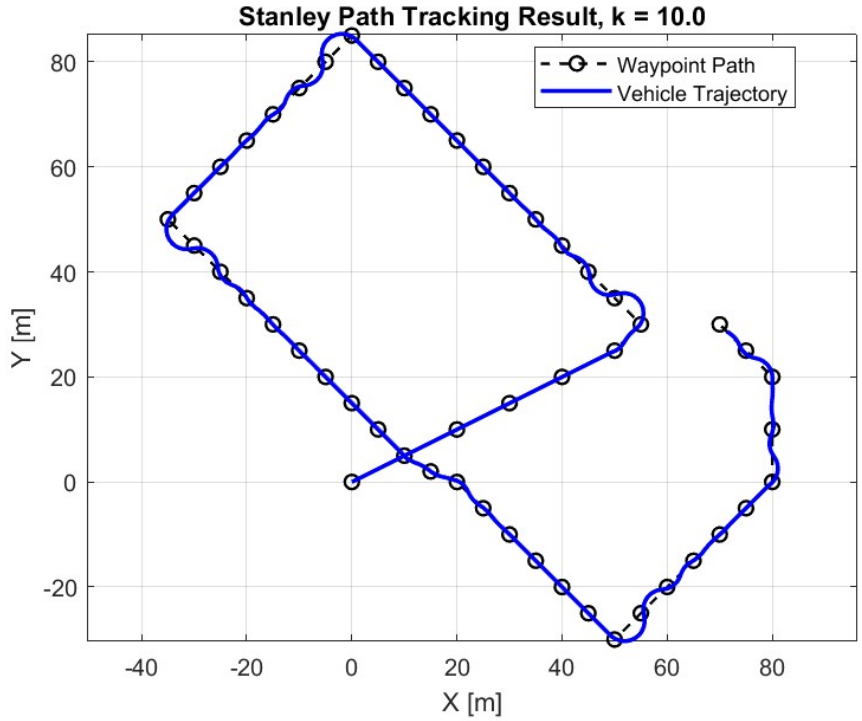


Figure 5: Path tracking result for $k = 10.0$.

For $k = 10.0$, the vehicle reacts more aggressively to cross-track error. Although the vehicle remains close to the path, small oscillations can be observed around some

sharp turns and waypoint transitions. This behavior is caused by the large lateral error correction term.

6.2.2 Combined Trajectory Comparison

Fig. 6 shows the trajectories for all gain values on the same plot.

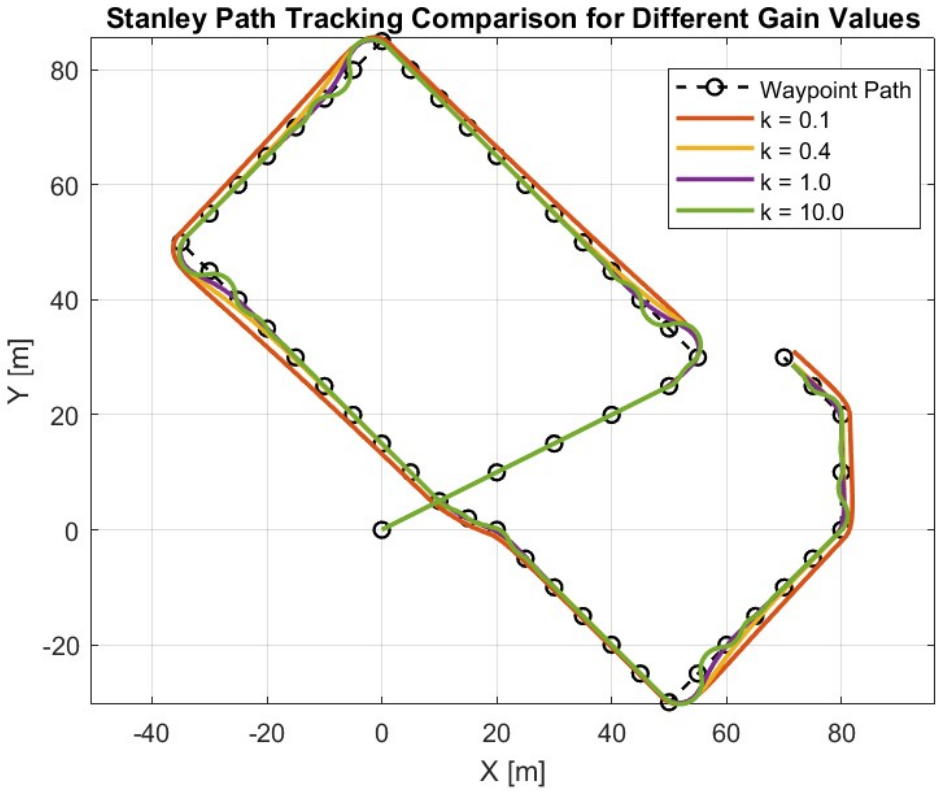


Figure 6: Combined trajectory comparison for different Stanley gain values.

The combined trajectory comparison shows that increasing k generally improves lateral convergence. However, an excessively large value of k makes the control response more aggressive. Therefore, moderate values such as $k = 0.4$ and $k = 1.0$ provide a better trade-off.

7 Error and Control Signal Analysis

7.1 Cross-Track Error

The front axle cross-track error e_f is shown in Fig. 7. This signal represents the lateral deviation of the front axle center from the active path segment.

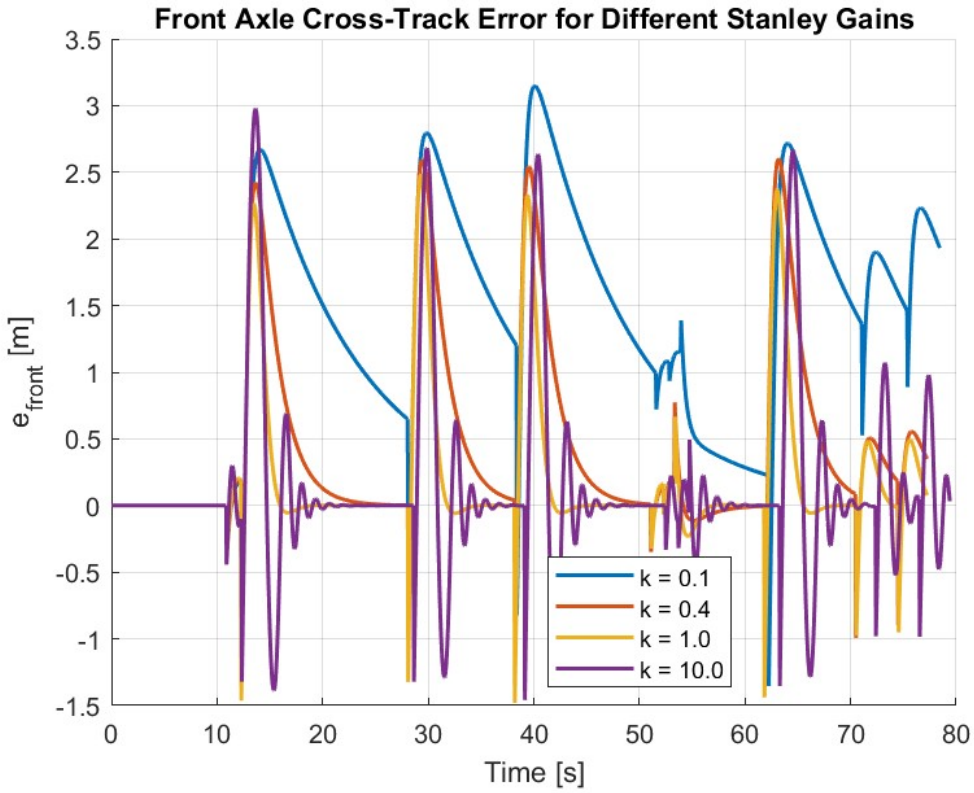


Figure 7: Front axle cross-track error comparison for different Stanley gain values.

For low gain values, the magnitude of e_f tends to be larger because the controller corrects lateral errors more slowly. As the gain increases, the vehicle converges to the path faster. However, high gain values may cause more oscillatory behavior in the error signal.

7.2 Heading Error

The heading error θ_e is shown in Fig. 8. This error corresponds to the difference between the path heading and the vehicle heading.

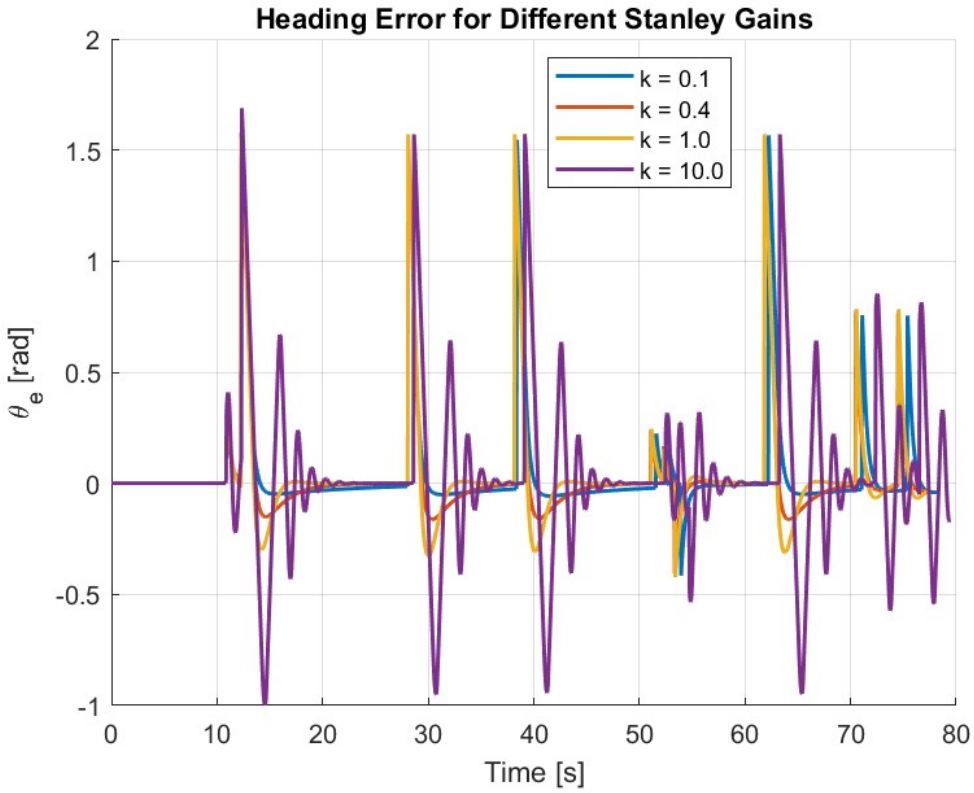


Figure 8: Heading error comparison for different Stanley gain values.

The heading error is especially affected by sharp changes in the waypoint path direction. Around sharp turns, θ_e changes rapidly. A moderate gain value allows the vehicle to correct the heading error without producing excessive oscillations.

7.3 Steering Angle

The steering command δ is shown in Fig. 9. The steering command is limited to

$$-0.5236 \leq \delta \leq 0.5236. \tag{21}$$

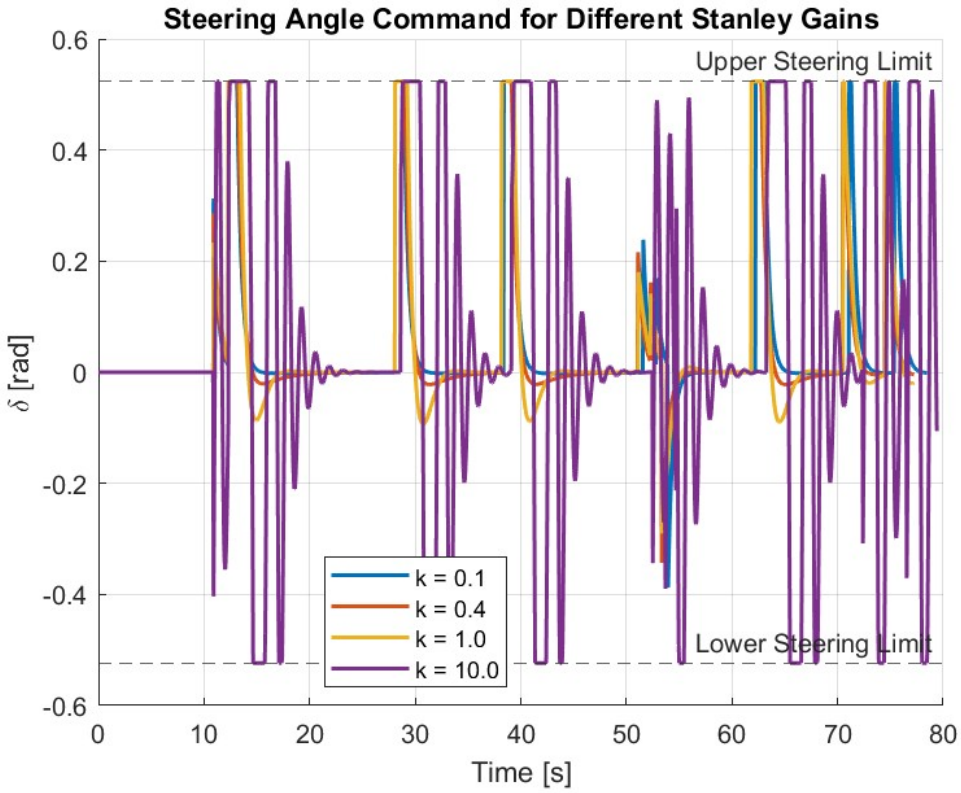


Figure 9: Steering angle command comparison for different Stanley gain values. The dashed lines indicate the steering saturation limits.

For $k = 10.0$, the steering command is expected to be more aggressive and may reach the saturation limits more frequently. This confirms that increasing the Stanley gain improves convergence but also increases control effort.

8 Performance Metrics

To quantitatively compare the effect of the Stanley gain, the following error metrics are calculated:

$$\text{Mean}|e_f| = \frac{1}{N}\sum_{j=1}^{N}|e_f(j)|, \tag{22}$$

$$\text{RMS}(e_f) = \sqrt{\frac{1}{N}\sum_{j=1}^{N}e_f(j)^2}, \tag{23}$$

$$\max|e_f| = \max_j |e_f(j)|. \tag{24}$$

The heading error and steering command metrics are calculated similarly.

Table 1: Cross-track error metrics for different Stanley gain values.

Stanley Gain k	Mean $\lvert e_f \rvert$ [m]	RMS(e_f) [m]	Max $\lvert e_f \rvert$ [m]
0.1	1.3465	1.6176	3.1480
0.4	0.4801	0.8461	2.6024
1.0	0.2550	0.6090	2.4873
10.0	0.3252	0.6995	2.9780

Table 2: Heading error metrics for different Stanley gain values.

Stanley Gain k	Mean $\lvert \theta_e \rvert$ [rad]	RMS(θ_e) [rad]	Max $\lvert \theta_e \rvert$ [rad]
0.1	0.0864	0.2414	1.5790
0.4	0.0938	0.2488	1.5713
1.0	0.0941	0.2548	1.5708
10.0	0.1791	0.3484	1.6887

Table 3: Steering command metrics for different Stanley gain values.

Stanley Gain k	Mean $\lvert \delta \rvert$ [rad]	Max $\lvert \delta \rvert$ [rad]
0.1	0.0488	0.5236
0.4	0.0551	0.5236
1.0	0.0635	0.5236
10.0	0.1904	0.5236

9 Discussion

The simulation results demonstrate the expected effect of the Stanley gain on path tracking performance. The gain k scales the cross-track error correction term in the steering command. Therefore, increasing k causes the vehicle to react more strongly to lateral deviation.

For $k = 0.1$, the controller response is smooth, but the lateral convergence is weak. The vehicle can follow the general path shape, but larger deviations appear around turns. This is because the cross-track error term is small compared to the heading error term.

For $k = 0.4$ and $k = 1.0$, the path tracking performance improves significantly. These values provide a better compromise between lateral accuracy and smooth steering behavior. The vehicle remains close to the waypoint path without producing excessive oscillations.

For $k = 10.0$, the controller becomes much more aggressive. The vehicle quickly reacts to lateral error, but this may result in oscillatory behavior near sharp turns and waypoint transitions. The steering angle may also reach the saturation limits more frequently. Therefore, although high gain can improve convergence, it may reduce smoothness and increase control effort.

Overall, the simulation results suggest that moderate Stanley gain values provide the most balanced performance for the given path and vehicle velocity.

10 Conclusion

In this homework, the Stanley path tracking algorithm was implemented in Simulink together with a kinematic bicycle vehicle model. The front axle center was used as the reference point for path tracking. The steering angle was limited to ± 0.5236 rad and the simulation was performed using a fixed-step solver with a sampling time of 0.05 s.

The nominal simulation with $k = 1.0$ showed that the vehicle successfully tracks the waypoint path. Then, different Stanley gain values were tested. The results showed that low gain values produce smoother but less accurate tracking, while very high gain values produce aggressive steering and may introduce oscillations. Among the tested values, $k = 0.4$ and $k = 1.0$ provided the best compromise between tracking accuracy and smooth control behavior.

A MATLAB Function Block Code

```
function [delta, seg_idx, e_front, theta_e, x_f, y_f] =
    stanley_controller(x_r, y_r, psi, k, v, L, waypoints)
% Stanley path tracking controller with front axle reference
    point.

persistent current_segment

if isempty(current_segment)
    current_segment = 1;
end

N = size(waypoints, 1);

if current_segment > N - 1
    current_segment = N - 1;
end

% Front axle point
x_f = x_r + L*cos(psi);
y_f = y_r + L*sin(psi);
pf = [x_f; y_f];

% Find closest segment ahead, but never go back
best_idx = current_segment;
best_dist = 1.0e9;

for i = current_segment:N-1

    p1 = waypoints(i, :)';
    p2 = waypoints(i+1, :)';

    seg_vec = p2 - p1;
    seg_len = norm(seg_vec);

    if seg_len < 1e-9
        continue;
    end

    t_hat = seg_vec / seg_len;
    rel_vec = pf - p1;
    s = dot(rel_vec, t_hat);

    s_clamped = min(max(s, 0), seg_len);
    p_proj = p1 + s_clamped*t_hat;

    dist = norm(pf - p_proj);

    if dist < best_dist
        best_dist = dist;
```

```
        best_idx = i;
    end
end

if best_idx >= current_segment
    current_segment = best_idx;
end

% Active segment
p1 = waypoints(current_segment, :)';
p2 = waypoints(current_segment + 1, :)';

seg_vec = p2 - p1;
seg_len = norm(seg_vec);

if seg_len < 1e-9
    delta = 0;
    seg_idx = current_segment;
    e_front = 0;
    theta_e = 0;
    return;
end

t_hat = seg_vec / seg_len;
rel_vec = pf - p1;
s = dot(rel_vec, t_hat);
dist_to_next = norm(pf - p2);

switch_distance = 2.0;

if (s >= seg_len || dist_to_next < switch_distance) &&
   current_segment < N - 1
    current_segment = current_segment + 1;

    p1 = waypoints(current_segment, :)';
    p2 = waypoints(current_segment + 1, :)';

    seg_vec = p2 - p1;
    seg_len = norm(seg_vec);
    t_hat = seg_vec / seg_len;

    rel_vec = pf - p1;
    s = dot(rel_vec, t_hat);
end

s_clamped = min(max(s, 0), seg_len);
p_proj = p1 + s_clamped*t_hat;

% Heading error
path_yaw = atan2(t_hat(2), t_hat(1));
theta_e = wrapToPiLocal(path_yaw - psi);
```

```
98
99  % Cross-track error
100 n_left = [-t_hat(2); t_hat(1)];
101 e_front = dot(pf - p_proj, n_left);
102
103 % Stanley control law
104 delta_unsat = theta_e - atan2(k*e_front, max(v, 0.1));
105
106 % Steering saturation
107 delta_max = 0.5236;
108
109 if delta_unsat > delta_max
110     delta = delta_max;
111 elseif delta_unsat < -delta_max
112     delta = -delta_max;
113 else
114     delta = delta_unsat;
115 end
116
117 seg_idx = current_segment;
118
119 end
120
121
122 function angle = wrapToPiLocal(angle)
123
124 while angle > pi
125     angle = angle - 2*pi;
126 end
127
128 while angle < -pi
129     angle = angle + 2*pi;
130 end
131
132 end
```

Listing 1: Stanley controller MATLAB Function block.

B Part (b) MATLAB Analysis Script

```
1  % Load parameters
2  params_hw3;
3
4  modelName = 'untitled3';
5  k_values = [0.1, 0.4, 1.0, 10.0];
6  Tfinal = 120;
7
8  psi0 = atan2(waypoints(2,2) - waypoints(1,2), ...
9               waypoints(2,1) - waypoints(1,1));
10
```

```
x_r0 = waypoints(1,1) - L*cos(psi0);
y_r0 = waypoints(1,2) - L*sin(psi0);

xfinal = waypoints(end,1);
yfinal = waypoints(end,2);

results = struct();

for i = 1:length(k_values)

    k = k_values(i);
    out = sim(modelName, 'StopTime', num2str(Tfinal));

    x_log = out.x_f_log;
    y_log = out.y_f_log;
    seg_log = out.seg_idx_log;
    e_log = out.e_front_log;
    theta_log = out.theta_e_log;
    delta_log = out.delta_log;

    if isa(x_log, 'timeseries')
        t = x_log.Time;
        x = x_log.Data;
        y = y_log.Data;
        seg_idx = seg_log.Data;
        e_front = e_log.Data;
        theta_e = theta_log.Data;
        delta = delta_log.Data;
    else
        t = x_log(:,1);
        x = x_log(:,2);
        y = y_log(:,2);
        seg_idx = seg_log(:,2);
        e_front = e_log(:,2);
        theta_e = theta_log(:,2);
        delta = delta_log(:,2);
    end

    dist_final = sqrt((x - xfinal).^2 + (y - yfinal).^2);
    final_tol = 2.0;

    idx_final = find(seg_idx >= size(waypoints,1)-1 & ...
                     dist_final < final_tol, 1, 'first');

    if isempty(idx_final)
        idx_plot = 1:length(x);
    else
        idx_plot = 1:idx_final;
    end

    results(i).k = k;
```

```
    results(i).t = t(idx_plot);
    results(i).x = x(idx_plot);
    results(i).y = y(idx_plot);
    results(i).seg_idx = seg_idx(idx_plot);
    results(i).e_front = e_front(idx_plot);
    results(i).theta_e = theta_e(idx_plot);
    results(i).delta = delta(idx_plot);

    results(i).mean_abs_e = mean(abs(results(i).e_front));
    results(i).rms_e = sqrt(mean(results(i).e_front.^2));
    results(i).max_abs_e = max(abs(results(i).e_front));

    results(i).mean_abs_theta = mean(abs(results(i).theta_e));
    results(i).rms_theta = sqrt(mean(results(i).theta_e.^2));
    results(i).max_abs_theta = max(abs(results(i).theta_e));

    results(i).mean_abs_delta = mean(abs(results(i).delta));
    results(i).max_abs_delta = max(abs(results(i).delta));
end
```

Listing 2: MATLAB script for gain comparison and signal analysis.